

Claude Code Prompt — MF Portfolio Import Prototype (CAS-first)

Copy everything below the line into Claude Code as your kickoff prompt.

Context

I'm building a prototype of the mutual-fund import module for a wealth-management platform (an MProfit competitor). MFCentral's CAS API was shut down by SEBI/AMFI in Sept 2025, so the industry-standard approach now is: **user uploads their own CAS (Consolidated Account Statement) PDF** → **we parse it** → **enrich with live NAV data** → **compute holdings, XIRR, and capital gains**. Build exactly that, end to end, with a minimal but clean UI.

Stack

- **Backend:** Python 3.11+, FastAPI, `casparseser` (open-source, MIT — parses CAMS & KFinTech detailed CAS PDFs into structured JSON), `httpx` for NAV fetching, `python-dateutil`.
- **Money math:** `decimal.Decimal` everywhere. NEVER float for amounts, units, NAVs, or gains. Units to 3 decimal places, amounts to 2, NAV to 4. This is non-negotiable.
- **DB:** SQLite via SQLAlchemy (prototype only; schema should be Postgres-portable — use Numeric column types, not Float).
- **Frontend:** Single-page React (Vite) or plain HTML+htmx — whichever is faster. Minimal: upload screen, processing state, portfolio dashboard. No auth for the prototype.
- **NAV enrichment:** mfapi.in (free, no key): `GET https://api.mfapi.in/mf/{scheme_code}` for full NAV history, `/latest` for current NAV. Cache responses locally (NAVs update once daily — do not hammer the API).

Core flow

1. **Upload:** User uploads a CAS PDF (CAMS "Detailed" CAS or MFCentral-generated detailed CAS) + the PDF password they set when requesting it. Show clear helper text explaining how to get one (camsonline.com → Statements → CAS, choose *Detailed*, period = since first investment).
2. **Parse:** Run `casparseser` on the file. Handle both CAMS and KFinTech layouts (the lib does this). Extract: investor info (name, email, PAN — mask PAN in UI as ABCDE****F), folios, schemes (with ISIN + AMFI scheme code where present), and every transaction

(date, description, amount, units, NAV, type: PURCHASE / PURCHASE_SIP / REDEMPTION / SWITCH_IN / SWITCH_OUT / DIVIDEND_PAYOUT / DIVIDEND_REINVEST / SEGREGATION / STAMP_DUTY / STT / MISC).

3. **Normalize & store:** One canonical **transactions** table. Map casparser's transaction types into our enum. Store raw parser output too (JSON column) for debugging. Dedupe on (folio, scheme, date, amount, units) so re-uploading an overlapping CAS doesn't double-count — report "n new, m duplicates skipped" to the user.
4. **Enrich:** For each scheme, resolve the AMFI scheme code (casparser usually provides it; if missing, fuzzy-match scheme name against mfapi.in's scheme list [GET https://api.mfapi.in/mf](https://api.mfapi.in/mf) and flag low-confidence matches for user confirmation in the UI — never silently guess). Pull latest NAV + historical NAV series.
5. **Compute** (all Decimal):
 - Current holdings per scheme: net units × latest NAV.
 - Invested amount (net of redemptions, FIFO cost basis).
 - **XIRR** per scheme and portfolio-wide: implement Newton-Raphson with bisection fallback on the cashflow series (outflows = purchases negative, inflows = redemptions + current value as terminal inflow). Handle edge cases: single cashflow, all-negative, non-convergence → show "—" not a crash.
 - **Capital gains (India):** FIFO lot matching. Equity funds: STCG ≤ 12 months, LTCG > 12 months, apply the ₹1.25L LTCG exemption note (display, don't tax-file). Debt funds bought after Apr 1 2023: all gains taxed at slab (no indexation) — just classify and label correctly. Include grandfathering note for pre-Feb 1 2018 equity purchases (use Jan 31 2018 NAV as substituted cost where beneficial — fetch that NAV from the historical series). Keep this as a clearly-labeled *estimate*, with a disclaimer that it is not tax advice.
6. **Dashboard UI:**
 - Portfolio summary card: current value, invested, absolute gain, XIRR.
 - Holdings table: scheme, folio, units, avg cost, current NAV, value, gain, XIRR. Sortable.
 - Allocation donut: equity vs debt vs hybrid vs other (derive from scheme category via mfapi.in metadata).
 - Per-scheme drill-down: transaction list + NAV line chart (historical series) with buy/sell markers.
 - Import history panel: file name, date, schemes found, txns added/skipped, and any low-confidence scheme matches awaiting confirmation.
7. **The "better than MProfit" touches** (cheap but differentiating):
 - Confidence-scored import review screen: show what we parsed *before* committing — user confirms, we save. (MProfit imports silently and users discover errors later.)
 - Value-at-date: historical portfolio valuation line computed from NAV history × unit balance over time.

- Clear empty/error states: wrong password, scanned/image-only PDF, summary-CAS-instead-of-detailed — each gets a specific, human error message telling the user exactly what to do.

Project structure

mf-import/

backend/

app/main.py # FastAPI app

app/parser.py # casparser wrapper + normalization

app/enrich.py # mfapi.in client with on-disk cache

app/calc.py # XIRR, FIFO gains, valuations (pure functions, Decimal)

app/models.py # SQLAlchemy models

app/schemas.py # Pydantic schemas

tests/ # pytest: XIRR known-answer tests, FIFO tests, parser fixture tests

frontend/ # minimal SPA

README.md # setup + how to get a CAS PDF

Rules

- Write `calc.py` as pure functions with pytest known-answer tests FIRST (e.g., XIRR of a simple SIP series verified against a spreadsheet value; FIFO gains on a hand-computed 4-transaction fixture). Then wire the rest.
- Uploaded PDFs: process in memory / temp dir, delete after parsing. Never log PAN, never store the PDF password.
- All API responses that include money: strings of Decimals, not JSON floats.
- If casparser fails on a file, fall back to reporting the failure cleanly with the parser's error — do NOT attempt to hand-roll PDF text extraction in v1.
- Commit in logical increments with clear messages.

Start by scaffolding the backend, writing `calc.py` + its tests, then the parser wrapper, then the API routes, then the frontend. Ask me before adding any dependency not listed above.

What I (the human) need to prepare

1. **A real test CAS:** request a *Detailed* CAS from camsonline.com (Statements → CAS → Detailed, period = all-to-date) to your email; you'll set a password — keep it. This is your primary test fixture. Get one KFintech-flavored CAS too if possible (via MFCentral's statement request) to test both layouts.
2. **No API keys needed** for the core build (mfapi.in is keyless). Optional: casparser.in free tier (10 credits/mo) if you want to benchmark your parse output against theirs; parse.bot key if you later want sector/holdings enrichment.
3. **Later (post-prototype):** shortlist an AA TSP (Setu / Finvu / Onemoney) and start the regulatory conversation early — FIU-relying-entity routes and timelines are the long pole for the real product.